

Low Latency and High Throughput Considerations for AI Applications

AI applications today power real-time recommendations, fraud detection, conversational agents, autonomous vehicles, and large-scale analytics. Two critical performance dimensions determine their effectiveness: **latency** and **throughput**.

- **Latency:** The time taken to process a single request end-to-end.
- **Throughput:** The number of requests or tasks the system can handle per unit of time.

Balancing these is essential, as AI workloads often need **fast responses** without sacrificing the ability to **scale to thousands or millions of users**.

1. Why Low Latency and High Throughput Matter

- **User Experience:** A chatbot taking 5 seconds to respond will frustrate users; 200 ms feels natural.
- **Business Value:** Real-time fraud detection must act within milliseconds before a transaction completes.
- **System Efficiency:** Higher throughput ensures cost efficiency when serving millions of inference requests.
- **Safety-Critical Scenarios:** Autonomous driving or medical imaging cannot tolerate delays.

2. Sources of Latency in AI Applications

Latency can accumulate from multiple stages:

1. Model Complexity

- Large models (e.g., GPT-class LLMs) have billions of parameters, leading to high compute requirements.
- Transformer architectures often scale quadratically with input sequence length.

2. Hardware Limitations

- GPUs/TPUs underutilized due to small batch sizes or memory bottlenecks.

- Inefficient kernel launches or I/O wait times.

3. Data Pipeline Bottlenecks

- Slow feature extraction or preprocessing (e.g., image resizing, tokenization).
- Remote storage access increasing I/O latency.

4. Network Overheads

- Latency from API calls, load balancers, or distributed inference setups.
- Serialization/deserialization overheads in data transfer.

5. Concurrency Handling

- Queuing delays under heavy load.
- Thread contention and locking issues.

3. Strategies for Low Latency

• Model Optimization

- Quantization, pruning, distillation to reduce model size.
- Use efficient architectures (e.g., MobileNet, LLaMA-optimized variants).
- Batch size tuning: smaller batches for interactive apps, larger for batch jobs.

• System-Level Optimizations

- GPU/TPU acceleration with optimized kernels (TensorRT, ONNX Runtime, FasterTransformer).
- Pinning critical data to GPU memory.
- Asynchronous pipelines to overlap preprocessing, inference, and postprocessing.

• Network and Deployment

- Edge inference to minimize network hops.
- Model sharding and caching for frequently used results.
- Low-latency RPC frameworks (gRPC, QUIC).

• Caching and Warm Starts

- Reuse cached embeddings or partial computations.

- Keep models "hot-loaded" to avoid cold start delays.

4. Strategies for High Throughput

- **Batching Requests**
 - Dynamic batching to combine multiple small requests.
 - Micro-batching for LLM inference.
- **Parallelization**
 - Data parallelism: replicate models across devices.
 - Pipeline parallelism: split model layers across devices.
 - Operator fusion: minimize kernel launch overheads.
- **Scalable Infrastructure**
 - Autoscaling on Kubernetes or serverless inference frameworks.
 - Load balancing with traffic-aware routing.
- **Resource Allocation**
 - Prioritization: critical requests get lower latency paths, bulk requests optimized for throughput.
 - Mixed precision (FP16/BF16) for more operations per second.

5. Trade-offs Between Latency and Throughput

- **Smaller batches = lower latency, but lower throughput**
- **Larger batches = higher throughput, but increased latency per request**

Example:

- A real-time voice assistant optimizes for **latency** (fast response per user).
- A recommendation engine serving millions of users may optimize for **throughput** (aggregate efficiency).

Designing the system requires aligning with the **application's SLA (Service Level Agreement)**.

6. Real-World Use Cases

1. **Conversational AI (Chatbots/Voice Assistants)**
 - Prioritize **low latency** for natural conversation.
 - Use optimized transformer variants and micro-batching.

2. Recommendation Systems (e-commerce, media)

- Prioritize **high throughput** to serve millions simultaneously.
- Use pre-computed embeddings and caching.

3. Fraud Detection

- Requires **both**: instant detection (low latency) and ability to scale across millions of transactions (high throughput).

4. Autonomous Vehicles

- Latency-critical (real-time environment perception).
- Offload heavy training to cloud, deploy lightweight models on edge.

7. Emerging Trends

• LLM Inference Optimization

- Speculative decoding, retrieval-augmented generation (RAG), and hybrid models to reduce inference time.

• Hardware Specialization

- AI accelerators (NVIDIA Tensor Cores, Graphcore IPUs, Cerebras Wafer-Scale Engine).

• Adaptive Serving

- Dynamic scaling based on real-time traffic and SLA-aware scheduling.

• Serverless Inference Platforms

- AWS Inferentia, Google TPU pods, Azure ML real-time endpoints.

8. Key Takeaways

- **Low Latency** ensures responsiveness and user satisfaction.
- **High Throughput** ensures scalability and cost efficiency.
- **Balancing both** requires optimizations at the model, system, and infrastructure levels.
- The right strategy depends on **application type, workload patterns, and SLA requirements**.